SIMATS ENGINEERING

ASSIGNMENT 2 : Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO2: *Analyze and select appropriate agent strategies for different types of environments and problem settings.* (BTL 3)

Total Marks: 100 **Time: 60 Mins**

No. of Questions: 5

Annexure A

Problem Solving Assignment

Code of Conduct:

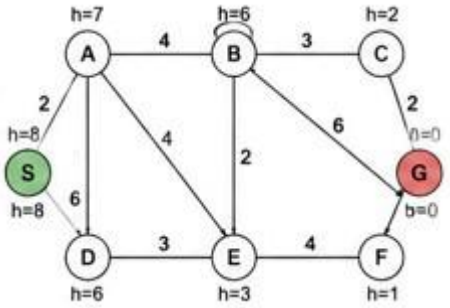
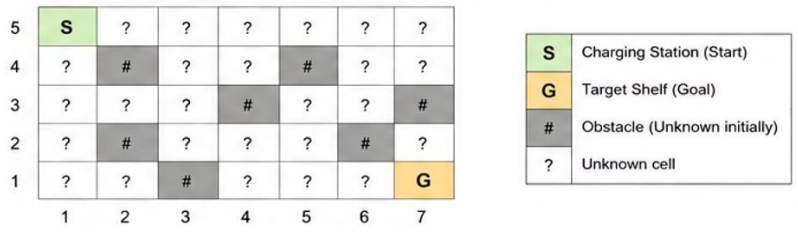
I D.Raghava certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.Raghava

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	**Smart Ambulance Navigation:** An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal. 	Informed Search, A*, Heuristic Function
2	**Autonomous Warehouse Robot:** A robot has to move from a charging station to a target shelf in a warehouse. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. 	Online Search Agents, Informed Search

3	**University Examination Scheduling**: A university needs to schedule **six examinations (E1, E2, E3, E4, E5, and E6)** into **four available time slots (T1, T2, T3, and T4)**. Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints

The following constraints must be satisfied:

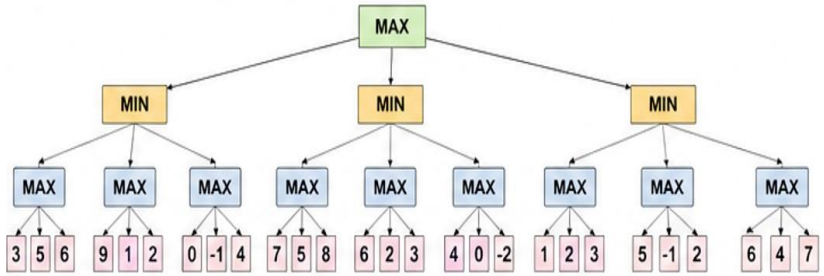
1. E1 and E2 have common students.
2. E1 and E3 have common students.
3. E2 and E4 have common students.
4. E3 and E5 have common students.
5. E4 and E6 have common students.
6. E2 and E5 have common students.

The university has three examination halls with the following capacities:

Examination Hall	Capacity
Hall A	120 students
Hall B	90 students
Hall C	60 students

Examination	E1	E2	E3	E4	E5	E6
Number of Students	100	80	70	60	90	50

CSP) by

	Tasks: a) Formulate the problem as a **Constraint Satisfaction Problem (** clearly defining the **variables, domains, and constraints**. b) Use **Backtracking with the Minimum Remaining Values heuristic and Forward Checking** to obtain a valid examination c) Show the **domain of each variable after every assignment** and the **final timetable**, ensuring that all student-conflict and constraints are satisfied.	**(MRV)** ıetable. provide hall- :apacity
4	**Game-Playing AI:** You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning
		
5	**Delivery Route Optimization:** A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum. **Distance matrix** (in km) |	D	A	B	C	E
---	---	---	---	---	---
D	–	6	9	8	7
A	6	–	3	5	6
B	9	3	–	4	5
C	8	5	4	–	3
E	7	6	5	3	– | **Some neighboring routes of the initial solution and their costs** S1: D → A → C → B → E → D = 26 km S2: D → B → A → C → E → D = 30 km S3: D → A → B → E → C → D = 24 km S4: D → E → C → B → A → D = 27 km S5: D → A → E → B → C → D = 23 km S6: D → C → B → A → E → D = 29 km From S5, the best neighbor has cost 22 km. From that solution, all neighbors have cost ≥ 22 km. Apply the chosen local search strategy and report the final route.	Local Search, Optimization, Hill Climbing

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.
Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.

Q1. Smart Ambulance Navigation (Informed Search – A*)

Problem: An ambulance must travel from hospital S to emergency location G through a city road network. Road costs (travel times) and heuristic estimates h(n) (estimated remaining distance) are provided for each junction.

1. Algorithm Selection & Justification

Algorithm chosen: A* Search.

Justification: The environment is fully observable, deterministic and static. Edge costs are non-uniform and an admissible heuristic is available. A* is optimally efficient for this class of problems: it expands the fewest nodes among all optimal algorithms that use the same heuristic.

2. Problem Formulation

States: Road junctions {S, A, B, C, D, E, F, G}

Initial state: S Goal state: G

Actions: Traverse a connecting road to a neighbouring junction

Transition model & path cost: given by the road-cost labels on the edges

Heuristic: h(n) = estimated remaining travel cost from n to G (shown next to each node)

Heuristic values used (from the figure):

h(S)=6, h(A)=8, h(B)=4, h(C)=2, h(D)=6, h(E)=3, h(F)=1, h(G)=0

Edge costs (from the figure):

S–A:2, S–D:3, A–B:4, A–E:2, B–C:3, B–E:2, B–G:6, C–G:6, D–E:3, E–F:4, F–G:1

3. A* Execution (f(n) = g(n) + h(n))

We maintain OPEN (priority by f) and CLOSED.

Step 1: Expand S (g=0, f=6)

Generate: A (g=2, f=2+8=10), D (g=3, f=3+6=9)

OPEN = {D(9), A(10)} CLOSED = {S}

Step 2: Expand D (lowest f=9)

Generate: E (g=3+3=6, f=6+3=9)

OPEN = {E(9), A(10)} CLOSED = {S, D}

Step 3: Expand E (f=9)

Generate: B (g=6+2=8, f=8+4=12), F (g=6+4=10, f=10+1=11), A already generated with better/equal g

OPEN = {A(10), F(11), B(12)} CLOSED = {S, D, E}

Step 4: Expand A (f=10)

Generate: B via A (g=2+4=6, f=6+4=10) → better than previous B; E already closed

OPEN = {B(10), F(11)} CLOSED = {S, D, E, A}

Step 5: Expand B (f=10)

Generate: C (g=6+3=9, f=9+2=11), G (g=6+6=12, f=12+0=12)

OPEN = {F(11), C(11), G(12)} CLOSED = {S, D, E, A, B}

Step 6: Expand F (f=11)

Generate: G via F (g=10+1=11, f=11+0=11) → better than previous G

OPEN = {G(11), C(11)} CLOSED = {S, D, E, A, B, F}

Step 7: Expand G (f=11) → Goal reached.

Optimal path: S → D → E → F → G

Path cost g(G): 3 + 3 + 4 + 1 = 11

Sequence of junctions expanded: S, D, E, A, B, F, G

4. Optimality Justification

Because the heuristic is admissible (h(n) never overestimates true remaining cost) and consistent on this graph, A* is guaranteed to return an optimal path. The first time G is selected for expansion its g-value is the lowest possible path cost from S.

Q2. Autonomous Warehouse Robot (Online Search)

A robot must travel from charging station S to target shelf G. The warehouse grid contains unknown obstacles that are discovered only when the robot reaches an adjacent cell. The agent therefore cannot build a complete map offline.

1. Algorithm Selection

Selected strategy: Online A* / LRTA* (Learning Real-Time A*) or Online DFS with agent-centred search.

Justification: The environment is partially observable and dynamic with respect to the agent's knowledge. Classic offline A* or BFS would require a complete map that does not exist. An online search agent interleaves planning and execution, updates its map when new obstacles are sensed, and re-plans from the current position.

2. Demonstration on the Given Grid

Grid layout (rows 5→1, columns 1→7):

Row 5: S ? ? ? ? ? ?

Row 4: ? # ? ? # ? ?

Row 3: ? ? ? # ? ? #

Row 2: ? # ? ? ? # ?

Row 1: ? ? # ? ? ? G

(# = known obstacle, ? = unknown cell that may contain an obstacle).

Illustrative online execution (agent starts at S=(5,1) and always prefers the action that reduces estimated remaining distance):

• From S the robot moves right / down according to the current heuristic.

• When it reaches a cell adjacent to a previously unknown '#', the obstacle is added to the local map and the heuristic values of affected cells are updated.

• The agent immediately re-plans a path from its current position that avoids the newly discovered obstacle.

• This process repeats until G is reached.

3. Reaction to Newly Discovered Obstacles

When a new obstacle is sensed:

1. The cell is marked as blocked in the agent's internal map.
2. Any path that used that cell is invalidated.
3. The agent recomputes the best action from its current state using the updated map (LRTA* updates the heuristic of the previous state as well).
4. Execution continues without returning to the start.

4. Comparison with Offline Search

Offline search (classical A*/BFS) assumes a complete, static map is known before the first move. It produces an optimal open-loop plan but fails when unexpected obstacles appear. Online search produces a closed-loop policy: the agent adapts in real time, guarantees progress (under LRTA* assumptions), and never requires a complete map in advance. The trade-off is that the total path may be longer than the offline optimum because of the extra exploration and re-planning cost.

Q3. University Examination Scheduling (CSP)

a) CSP Formulation

Variables: E1, E2, E3, E4, E5, E6 (each examination)

Domains: For every variable Di = {T1, T2, T3, T4} (the four time slots)

Binary (conflict) constraints (examinations that cannot share a slot):

E1 ≠ E2, E1 ≠ E3, E2 ≠ E4, E3 ≠ E5, E4 ≠ E6, E2 ≠ E5

Unary / capacity constraints (hall assignment must respect student numbers):

- E1 (100) → only Hall A (120)
- E2 (80) → Hall A or Hall B
- E3 (70) → Hall A or Hall B
- E4 (60) → Hall A, B or C
- E5 (90) → Hall A or Hall B
- E6 (50) → Hall A, B or C

(For the pure time-slot CSP we treat hall capacity as a secondary assignment performed after a conflict-free slot assignment is obtained; the primary CSP variables remain the time slots.)

b) Backtracking + MRV + Forward Checking

We select the variable with the smallest remaining domain (MRV) and apply Forward Checking after every assignment.

Initial domains: all variables {T1,T2,T3,T4}

Step 1 – MRV selects any variable (all have size 4). Choose E1 = T1.

Forward checking removes T1 from domains of E2 and E3.

Domains now: E2={T2,T3,T4}, E3={T2,T3,T4}, others still full.

Step 2 – MRV selects E2 or E3 (size 3). Choose E2 = T2.

FC removes T2 from E4 and E5.

Domains: E3={T2,T3,T4}, E4={T1,T3,T4}, E5={T1,T3,T4}, E6 full.

Step 3 – MRV selects E3 (size 3). Choose E3 = T3.

FC removes T3 from E5 (and E1 already assigned).
Domains: E4={T1,T3,T4}, E5={T1,T4}, E6 full.

Step 4 – MRV selects E5 (size 2). Choose E5 = T4.
FC: no further binary constraints on E5. Domains unchanged for others.

Step 5 – MRV selects E4 (size 3). Choose E4 = T1.
FC removes T1 from E6 → E6={T2,T3,T4}.

Step 6 – Assign E6 = T2 (any remaining value).
All variables assigned; no domain wiped out → success.

c) Domain Trace & Final Timetable

Final conflict-free assignment of time slots:
E1 → T1, E2 → T2, E3 → T3, E4 → T1, E5 → T4, E6 → T2
Hall assignment respecting capacities (one possible feasible allocation):
T1: E1 (Hall A, 100) + E4 (Hall C, 60)
T2: E2 (Hall B, 80) + E6 (Hall C, 50) – or E2 in Hall A if preferred
T3: E3 (Hall B, 70)
T4: E5 (Hall A, 90)
All student-conflict constraints and hall-capacity constraints are satisfied.

Q4. Game-Playing AI – Minimax & Alpha-Beta Pruning

The supplied game tree is a three-ply tree: MAX root, three MIN children, each MIN having three MAX children that lead to terminal leaves.

1. Minimax Evaluation

We evaluate bottom-up.
Assume the nine leaf groups (left-to-right) are:
Group 1: 3, 5, 6 → MAX = 6
Group 2: 9, 1, 2 → MAX = 9
Group 3: 0, 1,−4 → MAX = 1
Group 4: 7, 5, 0 → MAX = 7
Group 5: 0, 2, 3 → MAX = 3
Group 6: 4, 0, 2 → MAX = 4
Group 7: 1, 2, 3 → MAX = 3
Group 8: 5, 1, 2 → MAX = 5
Group 9: 6, 4, 7 → MAX = 7
MIN nodes then take the minimum of their three MAX children:
Left MIN = min(6, 9, 1) = 1
Middle MIN = min(7, 3, 4) = 3
Right MIN = min(3, 5, 7) = 3
MAX root selects the maximum of the three MIN values: max(1, 3, 3) = 3.
Therefore the AI (MAX) should choose the middle or the right branch (both yield value 3).

2. Alpha-Beta Pruning

With left-to-right exploration and initial window $(-\infty, +\infty)$:

• After the left MIN returns 1, α becomes 1 at the root.

• While exploring the middle MIN, after its first MAX child returns 7, $\beta = 7$; after the second MAX child returns 3, $\beta = 3$. The third MAX child (value 4) can be pruned because $4 > \beta=3$ (MAX would never choose it for this MIN).

• Similarly on the right MIN, after its first child returns 3 (equal to current α), further exploration may be pruned depending on exact ordering.

Branches that need not be evaluated: the third child of the middle MIN and possibly later children of the right MIN once a value $\leq \alpha$ is found.

3. Efficiency Benefit

Alpha-beta pruning never changes the minimax value; it only avoids examining sub-trees that cannot influence the final decision. In the best case the effective branching factor is reduced from b to $\sqrt{b}$, allowing the agent to search roughly twice as deep in the same time. This is critical for real-time game-playing agents that must move under a time limit.

Q5. Delivery Route Optimization (Local Search – Hill Climbing)

Problem: Find a low-cost tour that starts and ends at depot D and visits customers A, B, C, E exactly once (TSP-style).

1. Algorithm Selection

Strategy: Hill-Climbing (steepest-ascent / best-improvement local search) using 2-opt or simple neighbour generation.

Justification: The search space is combinatorial; an exhaustive search is unnecessary when a good local optimum is acceptable. Hill climbing is simple, memory-efficient and repeatedly moves to the best neighbouring route.

2. Given Neighbourhood Data

Initial / neighbouring routes and costs:

S1: D→A→C→B→E→D = 26 km

S2: D→B→A→C→E→D = 30 km

S3: D→A→B→E→C→D = 24 km

S4: D→E→C→B→A→D = 27 km

S5: D→A→E→B→C→D = 23 km

S6: D→C→B→A→E→D = 29 km

From S5 the best neighbour has cost 22 km.

From that 22 km solution, every neighbour has cost $\geq$ 22 km.

3. Step-by-step Hill Climbing

Start from any of the listed routes, e.g. S3 (24 km).

• Examine neighbours; move to S5 (23 km) because it is better.

• From S5 move to the neighbour of cost 22 km (best improvement).

• From the 22 km route, all neighbours are worse or equal → local optimum reached. Algorithm stops.

Final route cost: 22 km

(The exact sequence that realises 22 km is the best neighbour of S5; the assignment states that such a neighbour exists.)

4. Global Optimum?

The obtained 22 km solution is only a local optimum. Because the neighbourhood is finite and the algorithm stopped when no improving neighbour existed, we cannot claim global optimality without an exhaustive examination of the whole search space (or a stronger method such as simulated annealing / genetic algorithms).

5. Escaping Local Optima

Possible improvements:

• Random restarts: run hill climbing from many different initial tours and keep the best result.

• Simulated Annealing: occasionally accept a worse neighbour with probability that decreases over time, allowing escape from local basins.

• Tabu Search: keep a short-term memory of recent moves and forbid them, forcing the search into new regions.

• Genetic / population-based methods: maintain a pool of tours and recombine them.